# CA/TUCR/HTLA Inverse Bridge Formalization

*Discrete-to-Continuous Reduction for the 1024D Quinary HSL*

**Project Nexus v3.0**

**FORMAL EQUATIONS + BOUNDEDNESS PROOFS + NUMPY REFERENCE IMPLEMENTATION**

| | |
|---|---|
| **Document status** | Research-backed canonical candidate v1.0 |
| **Prepared for** | CA/TUCR/HTLA Project Nexus |
| **Architecture boundary** | 1024D quinary HSL -> 11D continuous ODE |
| **Primary outputs** | E_err in [0,1], RPE in [-1,1], Novelty in [0,1] |
| **Date** | July 18, 2026 |

*This report converts the completed deep-research result into a publication-ready engineering artifact. Exact coefficient values are architecture design defaults and require empirical calibration in the 40-step harness.*

# Abstract

This document closes the inverse half of the CA/TUCR/HTLA bridge. It defines how an expected 1024-dimensional quinary hypervector, an observed 1024-dimensional quinary hypervector, an immutable Value Anchor, and an optional warm-memory bank are reduced into the three bounded continuous scalars required by the 11D control ODE: general prediction error E_err, signed reward prediction error RPE, and Novelty. The construction is vacuum-safe, distinguishes inactive-coordinate mismatch from direct sign contradiction, preserves quinary magnitude, and avoids division-by-zero for all-vacuum inputs. The resulting interface is directly executable with NumPy and is suitable for insertion into the existing 40-step simulation harness.

**Core result**

The inverse bridge is a composite formalism: weighted quinary bit-plane mismatch for E_err, value-axis projection difference for RPE, and a local-manifold residual plus nearest-neighbor density term for Novelty.

# Contents


- 1. Architectural contract and boundary conditions
- 2. Vacuum-safe quinary plane encoding
- 3. General prediction error E_err
- 4. Reward prediction error RPE
- 5. Novelty as warm-manifold irreducibility
- 6. Boundedness and undefined-behavior proof
- 7. Runtime parameter contract
- 8. Bare-metal NumPy implementation
- 9. Validation and property tests
- 10. Integration into the 40-step harness
- 11. Ratification limits and open calibration questions
- References


# 1. Architectural Contract and Boundary Conditions

The already-compiled forward bridge takes E_err, RPE, and Novelty as continuous drives, evolves the 11D state, calculates k_allow through the 666D coupling layer, and applies a k-WTA sparsity wipeout to the 1024D quinary Hypercontext Salient Lattice (HSL). The inverse bridge must therefore produce bounded, deterministic scalars from the discrete substrate without inventing Euclidean semantics that erase the meaning of structural vacuum.

- Hypervectors have dimension D = 1024 and alphabet Q = {-2, -1, 0, 1, 2}.
- The symbol 0 is a Structural Vacuum, not an ordinary numeric midpoint.
- The active-masking invariant is P_A = 0 implies P_S = 0; inactive sign bits must never leak into similarity or error.
- A vacuum/non-vacuum mismatch must be distinguishable from a direct sign contradiction.
- E_err and Novelty must lie in [0,1]; RPE must lie in [-1,1].
- All-vacuum vectors, all-vacuum anchors, empty memory banks, and undersized neighborhoods must have defined behavior.

**Literature-to-architecture boundary**

The literature supports Hamming/popcount similarity for discrete hypervectors, normalized dot-product or cosine families for integer hypervectors, and nearest-neighbor/subspace residual methods for novelty detection. The exact plane weights and blend coefficients below are Project Nexus design constants, not universal theorems.

# 2. Vacuum-Safe Quinary Plane Encoding

For each coordinate x_i in Q, define three binary planes: activity A, sign S, and high-magnitude U.

$$a_i(x) = 1[x_i \neq 0] \tag{1}$$

$$s_i(x) = 1[x_i < 0] \tag{2}$$

$$u_i(x) = 1[|x_i| = 2] \tag{3}$$

| Quinary symbol | Activity A | Sign S | Strength U |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| +1 | 1 | 0 | 0 |
| -1 | 1 | 1 | 0 |

| | | | |
|---|---|---|---|
| +2 | 1 | 0 | 1 |
| -2 | 1 | 1 | 1 |

The signed-strength embedding reconstructs the exact quinary symbol while gating all sign and magnitude semantics through activity:

$$q_i(H) = (1 + u_i(H))\, a_i(H)\, (1 - 2\, s_i(H)) \quad in \{-2,-1,0,1,2\} \tag{4}$$

Because a_i = 0 forces q_i = 0, inactive coordinates cannot contribute stale sign bits. This is the inverse-bridge form of the P_A = 0 implies P_S = 0 invariant.

# 3. General Prediction Error E_err

E_err is value-neutral. It measures structured mismatch between H_exp and H_act on the active support union, not geometric distance in R^1024.

## 3.1 Per-coordinate cost

For expected and actual planes, define the coordinate cost c_i:

$$c_i = \alpha\, |a_i^{exp} - a_i^{act}| + \alpha_2\, |a_i^{exp} - a_i^{act}|\, \max(u_i^{exp}, u_i^{act}) + \beta\, a_i^{exp}\, a_i^{act}\, |s_i^{exp} - s_i^{act}| + \gamma\, a_i^{exp}\, a_i^{act}\, |u_i^{exp} - u_i^{act}| \tag{5}$$

Reference coefficients:

$$\alpha = 1/4, \quad \alpha_2 = 1/8, \quad \beta = 1/2, \quad \gamma = 1/8 \tag{6}$$

$$c_{max} = \max(\alpha + \alpha_2, \beta + \gamma) = 5/8 \tag{7}$$

## 3.2 Active-union normalization

$$n_{union} = \max(1, \sum_i \max(a_i^{exp}, a_i^{act})) \tag{8}$$

$$E_{err} = [\sum_i c_i] / [c_{max}\, n_{union}] \tag{9}$$

The equivalent bit-plane popcount expression is:

$$E_{err} = \{ \alpha\, popcnt(A_{exp} \oplus A_{act}) + \alpha_2\, popcnt[(A_{exp} \oplus A_{act}) \wedge (U_{exp} \vee U_{act})] + \beta\, popcnt[(A_{exp} \wedge A_{act}) \wedge (S_{exp} \oplus S_{act})] + \gamma\, popcnt[(A_{exp} \wedge A_{act}) \wedge (U_{exp} \oplus U_{act})] \} / \{ c_{max}\, \max[1, popcnt(A_{exp} \vee A_{act})] \} \tag{10}$$

## 3.3 Mismatch semantics

| Expected | Actual | Interpretation | Normalized cost |
|---|---|---|---|
| 0 | +/-1 | activity mismatch | 0.4 |
| 0 | +/-2 | strong activity mismatch | 0.6 |
| +1 | -1 | direct sign contradiction | 0.8 |
| +1 | +2 | magnitude-only mismatch | 0.2 |
| +1 | -2 | sign + magnitude contradiction | 1.0 |

Joint vacuums do not inflate agreement because the denominator uses the union of active coordinates rather than all 1024 positions.

# 4. Reward Prediction Error RPE

RPE adds value judgment. It compares how the expected and actual hypervectors project onto an immutable 1024D Value Anchor H_val.

## 4.1 Value-axis projection

$$V(H \mid H_{val}) = \langle q(H), q(H_{val}) \rangle / [2\, \|q(H_{val})\|_1 + \epsilon] \tag{11}$$

The normalized dot projection is primary because it preserves the extra salience of +/-2 relative to +/-1. A pure cosine would erase that intensity when vectors are collinear.

### 4.2 Signed error

$$RPE = clip( [V(H_act \mid H_val) - V(H_exp \mid H_val)] / 2, -1, 1 ) \quad (12)$$

- RPE > 0: the actual state is better than expected along the value axis.
- RPE < 0: the actual state is worse than expected along the value axis.
- RPE approximately 0 with large E_err: the mismatch is structurally important but value-neutral.
- If H_val is entirely vacuum, RPE is defined as 0; no value axis exists, so the bridge must not invent one.

### 4.3 Optional cosine diagnostic

$$V_cos(H \mid H_val) = \langle q(H), q(H_val) \rangle / [\|q(H)\|_2 \|q(H_val)\|_2 + epsilon] \quad (13)$$

V_cos can be logged as a diagnostic, but it should not replace Equation 11 in the primary runtime path unless quinary intensity is intentionally discarded.

## 5. Novelty as Warm-Manifold Irreducibility

Novelty is not simply prediction error. It asks whether H_act is reducible to the currently warm cognitive context. The warm manifold is represented by a memory bank W = {M_1,...,M_N}.

### 5.1 Local density deficit

$$D_j = E_err(M_j, H_act) \quad (14)$$

$$rho_K(H_act) = (1/K) \sum_{M \in N_K(H_act)} E_err(M, H_act) \quad (15)$$

rho_K is the mean distance to the K nearest warm memories and is already bounded in [0,1].

### 5.2 Orthogonal residual to the local manifold

$$mu = (1/K) \sum_{M \in N_K} q(M) \quad (16)$$

Let U_r contain the top r right-singular vectors of the centered K-neighbor matrix. Then:

$$eta_perp(H_act) = \|(I - U_r U_r^T)(q(H_act)-mu)\|_2^2 / [\|q(H_act)-mu\|_2^2 + epsilon] \quad (17)$$

### 5.3 Composite novelty scalar

$$Novelty = lambda\ eta_perp(H_act) + (1-lambda)\ rho_K(H_act), \quad lambda = 1/2 \quad (18)$$

- rho_K answers: Is the state far from warm memories?
- eta_perp answers: Is the state off the local manifold even when nearby memories exist?
- When too few neighbors exist for a subspace, the implementation falls back to rho_K.
- When no warm bank is supplied, the runtime uses {H_exp} as a one-point fallback manifold.

## 6. Boundedness and Undefined-Behavior Proof

### 6.1 E_err

Each coordinate cost satisfies 0 <= c_i <= c_max, and nonzero costs occur only on the active union. Therefore:

$$0 <= \sum_i c_i <= c_max\ n_union \Rightarrow 0 <= E_err <= 1 \quad (19)$$

If both vectors are all vacuum, n_union is replaced by 1 while the numerator is 0, so E_err = 0.

### 6.2 RPE

Since |q_i(H)| <= 2:

$$|\langle q(H), q(H_val) \rangle| <= 2 \|q(H_val)\|_1 \quad (20)$$

$$-1 <= V(H \mid H_val) <= 1 \quad (21)$$

$$-1 <= [V_act - V_exp]/2 <= 1 \quad (22)$$

If the Value Anchor is all vacuum, the implementation returns 0 before division.

### 6.3 Novelty

rho_K is an average of [0,1] distances. eta_perp is a residual-energy fraction under an orthonormal SVD basis, so it also lies in [0,1]. A convex combination remains in [0,1].

**Fail-safe conventions**

All-vacuum expected/actual -> E_err = 0. Null Value Anchor -> RPE = 0. Empty warm memory -> Novelty = 0 for all-vacuum actual, otherwise 1. Too few neighbors -> use the nearest-neighbor term only.

## 7. Runtime Parameter Contract

| Parameter | Default | Range / constraint | Role |
|---|---|---|---|
| alpha_occ | 0.25 | >= 0 | vacuum/non-vacuum mismatch |
| alpha_occ_u2 | 0.125 | >= 0 | extra charge when missing activity is magnitude 2 |
| beta_sign | 0.50 | >= 0 | direct sign contradiction |
| gamma_mag | 0.125 | >= 0 | magnitude 1 vs 2 mismatch |
| k | 8 | 1 <= k <= N | nearest warm memories |
| subspace_rank | 4 | 0 <= r < k | local manifold rank |
| manifold_weight lambda | 0.5 | 0 <= lambda <= 1 | orthogonal residual vs density blend |
| epsilon | 1e-12 | small positive | numerical guard |

These defaults encode the qualitative ordering requested by the architecture. They should be versioned in the manifest and calibrated against empirical trajectories rather than silently changed in code.

## 8. Bare-Metal NumPy Reference Implementation

The implementation below uses only NumPy. It validates the quinary alphabet before casting, preserves the 1024D shape contract, and returns Python floats ready for the 11D ODE.

```
from __future__ import annotations

import numpy as np

ALPHA_OCC = 0.25
ALPHA_OCC_U2 = 0.125
BETA_SIGN = 0.50
GAMMA_MAG = 0.125
C_MAX = max(ALPHA_OCC + ALPHA_OCC_U2, BETA_SIGN + GAMMA_MAG)
EPS = 1e-12
QUINARY = np.array([-2, -1, 0, 1, 2], dtype=np.int8)


def _validate_quinary_vector(
    x: np.ndarray,
    name: str = "vector",
    expected_dim: int | None = None,
) -> np.ndarray:
    arr = np.asarray(x)
    if arr.ndim != 1:
        raise ValueError(f"{name} must be a 1D array; got shape {arr.shape}.")
    if expected_dim is not None and arr.shape[0] != expected_dim:
        raise ValueError(
            f"{name} must have length {expected_dim}; got {arr.shape[0]}."
        )
    if not np.all(np.isfinite(arr)):
        raise ValueError(f"{name} contains NaN or infinity.")
    if not np.all(arr == np.rint(arr)):
        raise ValueError(f"{name} contains non-integral values.")
    arr = arr.astype(np.int8, copy=False)
    if not np.all(np.isin(arr, QUINARY)):
        raise ValueError(f"{name} contains values outside {{-2,-1,0,1,2}}.")
    return arr


def _validate_quinary_matrix(
    x: np.ndarray,
```

```
    name: str,
    expected_dim: int,
) -> np.ndarray:
    arr = np.asarray(x)
    if arr.ndim != 2:
        raise ValueError(f"{name} must be a 2D array; got shape {arr.shape}.")
    if arr.shape[1] != expected_dim:
        raise ValueError(
            f"{name} must have shape (N, {expected_dim}); got {arr.shape}."
        )
    if arr.shape[0] == 0:
        return arr.astype(np.int8)
    if not np.all(np.isfinite(arr)) or not np.all(arr == np.rint(arr)):
        raise ValueError(f"{name} must contain finite integer quinary values.")
    arr = arr.astype(np.int8, copy=False)
    if not np.all(np.isin(arr, QUINARY)):
        raise ValueError(f"{name} contains values outside {{-2,-1,0,1,2}}.")
    return arr


def _planes(h: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """Return activity A, sign S, magnitude-2 U, and reconstructed quinary q."""
    h = _validate_quinary_vector(h)
    A = (h != 0).astype(np.uint8)
    S = (h < 0).astype(np.uint8)
    U = (np.abs(h) == 2).astype(np.uint8)
    q = (
        (1 + U.astype(np.int8))
        * A.astype(np.int8)
        * (1 - 2 * S.astype(np.int8))
    ).astype(np.int8)
    return A, S, U, q


def prediction_error(H_exp: np.ndarray, H_act: np.ndarray) -> float:
    """Weighted plane-popcount prediction error in [0,1]."""
    H_exp = _validate_quinary_vector(H_exp, "H_exp")
    H_act = _validate_quinary_vector(H_act, "H_act", H_exp.shape[0])
    Ae, Se, Ue, _ = _planes(H_exp)
    Aa, Sa, Ua, _ = _planes(H_act)

    occ = np.bitwise_xor(Ae, Aa).astype(np.float64)
    joint = np.bitwise_and(Ae, Aa).astype(np.float64)
    sign = joint * np.bitwise_xor(Se, Sa).astype(np.float64)
    mag = joint * np.bitwise_xor(Ue, Ua).astype(np.float64)
    missing_u2 = occ * np.bitwise_or(Ue, Ua).astype(np.float64)

    cost = (
        ALPHA_OCC * occ
        + ALPHA_OCC_U2 * missing_u2
        + BETA_SIGN * sign
        + GAMMA_MAG * mag
    )
    support_union = int(np.bitwise_or(Ae, Aa).sum())
    denom = C_MAX * max(1, support_union)
    return float(np.clip(cost.sum() / denom, 0.0, 1.0))


def value_projection(H: np.ndarray, H_val: np.ndarray) -> float:
    """Intensity-preserving value-axis projection in [-1,1]."""
    H = _validate_quinary_vector(H, "H")
    H_val = _validate_quinary_vector(H_val, "H_val", H.shape[0])
    _, _, _, q = _planes(H)
    _, _, _, qv = _planes(H_val)
    q = q.astype(np.float64)
    qv = qv.astype(np.float64)
    denom = 2.0 * np.abs(qv).sum()
    if denom < EPS:
        return 0.0
    return float(np.clip(np.dot(q, qv) / denom, -1.0, 1.0))


def reward_prediction_error(
    H_exp: np.ndarray,
    H_act: np.ndarray,
    H_val: np.ndarray,
) -> float:
    """Signed reward prediction error in [-1,1]."""
    v_exp = value_projection(H_exp, H_val)
    v_act = value_projection(H_act, H_val)
    return float(np.clip(0.5 * (v_act - v_exp), -1.0, 1.0))


def novelty_score(
    H_act: np.ndarray,
    warm_memory: np.ndarray | None = None,
```

```
    H_exp_fallback: np.ndarray | None = None,
    k: int = 8,
    subspace_rank: int = 4,
    manifold_weight: float = 0.5,
) -> float:
    """Warm-manifold novelty in [0,1]."""
    H_act = _validate_quinary_vector(H_act, "H_act")
    D = H_act.shape[0]
    if not 0.0 <= manifold_weight <= 1.0:
        raise ValueError("manifold_weight must lie in [0,1].")
    if k < 1 or subspace_rank < 0:
        raise ValueError("k must be >= 1 and subspace_rank must be >= 0.")

    if warm_memory is None:
        if H_exp_fallback is None:
            W = H_act.reshape(1, -1)
        else:
            fallback = _validate_quinary_vector(
                H_exp_fallback, "H_exp_fallback", D
            )
            W = fallback.reshape(1, -1)
    else:
        W = _validate_quinary_matrix(warm_memory, "warm_memory", D)

    if W.shape[0] == 0:
        return 0.0 if np.all(H_act == 0) else 1.0

    dists = np.array(
        [prediction_error(mem, H_act) for mem in W], dtype=np.float64
    )
    k_eff = min(k, W.shape[0])
    nn_idx = np.argsort(dists, kind="stable")[:k_eff]
    N = W[nn_idx].astype(np.float64)
    rho_k = float(np.clip(dists[nn_idx].mean(), 0.0, 1.0))

    mu = N.mean(axis=0)
    Z = N - mu
    if N.shape[0] < 2 or np.allclose(Z, 0.0):
        eta_perp = rho_k
    else:
        _, _, VT = np.linalg.svd(Z, full_matrices=False)
        r = min(subspace_rank, VT.shape[0], N.shape[0] - 1)
        if r <= 0:
            eta_perp = rho_k
        else:
            basis = VT[:r].T
            z = H_act.astype(np.float64) - mu
            denom = float(np.dot(z, z))
            if denom < EPS:
                eta_perp = 0.0
            else:
                z_parallel = basis @ (basis.T @ z)
                residual = z - z_parallel
                eta_perp = float(
                    np.clip(np.dot(residual, residual) / denom, 0.0, 1.0)
                )

    novelty = manifold_weight * eta_perp + (1.0 - manifold_weight) * rho_k
    return float(np.clip(novelty, 0.0, 1.0))


def inverse_bridge_scalars(
    H_exp: np.ndarray,
    H_act: np.ndarray,
    H_val: np.ndarray,
    warm_memory: np.ndarray | None = None,
    k: int = 8,
    subspace_rank: int = 4,
    manifold_weight: float = 0.5,
) -> tuple[float, float, float]:
    """Return (E_err, RPE, Novelty) for direct use by the 11D ODE."""
    H_exp = _validate_quinary_vector(H_exp, "H_exp", 1024)
    H_act = _validate_quinary_vector(H_act, "H_act", 1024)
    H_val = _validate_quinary_vector(H_val, "H_val", 1024)

    E_err = prediction_error(H_exp, H_act)
    RPE = reward_prediction_error(H_exp, H_act, H_val)
    Novelty = novelty_score(
        H_act,
        warm_memory=warm_memory,
        H_exp_fallback=H_exp,
        k=k,
        subspace_rank=subspace_rank,
        manifold_weight=manifold_weight,
    )
    return E_err, RPE, Novelty
```

# 9. Validation and Property Tests

The following checks should run before integration and on every coefficient change.

```
def _single(value: int, index: int = 0) -> np.ndarray:
    x = np.zeros(1024, dtype=np.int8)
    x[index] = value
    return x


def smoke_tests() -> None:
    z = np.zeros(1024, dtype=np.int8)
    p1 = _single(+1)
    n1 = _single(-1)
    p2 = _single(+2)

    assert prediction_error(z, z) == 0.0
    assert np.isclose(prediction_error(z, p1), 0.4)
    assert np.isclose(prediction_error(z, p2), 0.6)
    assert np.isclose(prediction_error(p1, n1), 0.8)
    assert np.isclose(prediction_error(p1, p2), 0.2)

    assert reward_prediction_error(z, z, z) == 0.0
    assert reward_prediction_error(z, p2, p2) > 0.0
    assert reward_prediction_error(p2, z, p2) < 0.0

    rng = np.random.default_rng(7)
    W = rng.integers(-2, 3, size=(16, 1024), dtype=np.int8)
    e, r, n = inverse_bridge_scalars(W[0], W[1], W[2], W)
    assert 0.0 <= e <= 1.0
    assert -1.0 <= r <= 1.0
    assert 0.0 <= n <= 1.0


def fuzz_bounds(trials: int = 10_000, seed: int = 19) -> None:
    rng = np.random.default_rng(seed)
    for _ in range(trials):
        exp = rng.integers(-2, 3, size=1024, dtype=np.int8)
        act = rng.integers(-2, 3, size=1024, dtype=np.int8)
        val = rng.integers(-2, 3, size=1024, dtype=np.int8)
        e = prediction_error(exp, act)
        r = reward_prediction_error(exp, act, val)
        assert 0.0 <= e <= 1.0
        assert -1.0 <= r <= 1.0
```

- Identity: E_err(H,H) = 0 and RPE(H,H,H_val) = 0.
- Symmetry: E_err(X,Y) = E_err(Y,X).
- Vacuum safety: changing inactive sign storage cannot affect any scalar.
- Monotonic support: adding independent mismatches cannot reduce the unnormalized error numerator.
- Bound fuzzing: random quinary vectors never escape the required intervals.
- Novelty calibration: known warm states should cluster near 0; held-out or synthetically orthogonal states should trend toward 1.

# 10. Integration into the 40-Step Harness

At every simulation step t:

1. Obtain H_exp(t) from the VAGAN prediction path.
2. Encode the current observation as H_act(t) in the same quinary basis and label ordering.
3. Load the immutable H_val and the current warm-memory bank W(t).
4. Call inverse_bridge_scalars to produce E_err(t), RPE(t), and Novelty(t).
5. Feed the three scalars into the 11D ODE without additional rescaling unless the ODE contract explicitly specifies it.
6. Compute k_allow through the forward bridge and apply the active-mask-first k-WTA wipeout.
7. Log the three scalars, active union size, nearest-neighbor distances, subspace rank, and selected k_allow for each step.

**Do not silently normalize twice**

The inverse bridge already emits bounded scalars. Any extra sigmoid, tanh, min-max scaling, or division in the ODE input path changes the ratified semantics and must be treated as a new interface version.

## Recommended trace record

```
{
    "step": t,
    "E_err": E_err,
    "RPE": RPE,
    "Novelty": Novelty,
```

```
    "active_union": int(np.count_nonzero((H_exp != 0) | (H_act != 0))),
    "warm_count": 0 if warm_memory is None else int(len(warm_memory)),
    "knn_k": k,
    "subspace_rank": subspace_rank,
    "manifold_weight": manifold_weight,
    "k_allow": k_allow,
    "inverse_bridge_version": "1.0"
}
```

# 11. Ratification Limits and Open Calibration Questions

- The metric families are literature-grounded; the exact alpha, beta, gamma, lambda, K, and rank values are architecture-specific hypotheses.
- The Value Anchor must have a documented construction and governance rule. A mutable anchor would turn RPE into policy drift.
- The warm-memory bank must define admission, eviction, and age weighting. Otherwise Novelty can be manipulated by memory saturation.
- The local PCA approximation is linear. Curved or disconnected warm manifolds may require graph-geodesic or kernel residual diagnostics later.
- E_err is a bounded dissimilarity score. Unless triangle inequality is separately proven for the selected weights, do not call it a formal metric.
- Empirical calibration must test whether sign contradictions should be magnitude-weighted more strongly than the current defaults.
- The 40-step harness should include adversarial all-vacuum, dense, contradictory, anchor-null, memory-empty, and neighborhood-degenerate cases.

**Canonical status recommendation**

Lock the interface and equations as Inverse Bridge v1.0-RC1, not as an immutable theorem. Promote to v1.0 only after the 40-step harness verifies bounds, monotonicity, vacuum invariance, and useful scalar dynamics under representative workloads.

# References

**[1] Project Nexus architecture source.** CA/TUCR/HTLA Continuous-to-Discrete Bridge Formalization and empirical bridge test context, supplied by the project owner.

**[2] Kleyko et al. (2023).** A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part I: Models and Data Transformations. ACM Computing Surveys. DOI: 10.1145/3538531.

**[3] Schlegel et al. (2022).** A comparison of vector symbolic architectures. Artificial Intelligence Review 55, 4523-4555. DOI: 10.1007/s10462-021-10110-3.

**[4] Pimentel et al. (2014).** A review of novelty detection. Signal Processing 99, 215-249. DOI: 10.1016/j.sigpro.2013.12.026.

**[5] Breunig et al. (2000).** LOF: Identifying Density-Based Local Outliers. ACM SIGMOD, 93-104. DOI: 10.1145/335191.335388.

**[6] Zhu and Xue (2017).** On the orthogonal distance to class subspaces for high-dimensional data classification. Information Sciences 417, 464-479.

**[7] Dunia et al. (1998).** Subspace approach to multidimensional fault identification and reconstruction. AIChE Journal 44(8), 1813-1831. DOI: 10.1002/aic.690440812.

**[8] Starkweather et al. (2017).** Dopamine reward prediction errors reflect hidden-state inference across time. Nature Neuroscience 20, 581-589. DOI: 10.1038/nn.4520.

**[9] Kahnt (2025).** The curious case of dopaminergic prediction errors and learning associative information beyond value. Nature Reviews Neuroscience. DOI: 10.1038/s41583-024-00898-8.

## Selected source links

- IBM HDC/VSA Survey Part I
- Vector Symbolic Architecture comparison
- Novelty detection review
- LOF original publication record
- Reward prediction error paper
- Orthogonal class-subspace distance

**END OF REPORT**